

Course Outline: Training a Classification Model

Title: Training a Classification Model **Skill:** __ — Skill x: Model selection and training **Learnpack:** + Cómo entrenar un modelo de Clasificación (predice una categoría) **File:** s__-w17d50-como-entrenar-un-modelo-de-clasificacion **Prerequisite:** Preparing Data for Model Training (ETL pipeline, cleaned datasets) **Target Audience:** Beginner developers who have prepared a dataset and want to train their first classifier **Total Lessons:** 10 **Format:** Mixed (10% hands-on)

Course Description

This course teaches how to train a machine learning classification model — a model that predicts a category — using Python and scikit-learn. Students already know what classification is conceptually (from the Prediction Models learnpack) and how to prepare data (from the ETL learnpack). Here they learn the mechanics: splitting data into train and test sets, choosing an algorithm, fitting the model, making predictions, and evaluating with classification-specific metrics. By the end, students have run a complete supervised learning training cycle.

Course Philosophy

This course emphasizes:

- Learning by doing the full training loop from scratch
 - Understanding WHY each step exists, not just how to run it
 - Building intuition for what "good" model performance looks like
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - How Classification Training Works	2
02 - Selecting and Training	3
03 - Evaluating Performance	2
04 - Assessment	1
05 - Outro	1
Total	10

00.0 Teaching a Machine to Categorize 

Learning Objectives:

- Explain what supervised learning is and why training is necessary
- Connect the ETL pipeline output to the classifier training input

Content Outline:

- What "training" a model means: finding patterns from labeled examples
- The full supervised learning loop: prepare → split → train → evaluate → improve
- Why we need labeled data to train a classifier
- Where this course fits: the ETL learnpack produced a clean dataset; this course uses it

Transitions: This lesson frames everything that follows. Section 01 starts with how the training loop actually works.

Key Principles:

- A classifier learns from examples: show it enough labeled data and it learns decision boundaries
- No training = no learning: every classifier must be fit to data before it can predict anything

Examples:

- Email spam filter: trained on thousands of labeled emails (spam / not spam) → predicts label for new emails
 - Iris species classifier: trained on 150 labeled flower measurements → predicts species from new measurements
-

01.0 How Classification Training Works

Learning Objectives:

- Describe the supervised learning cycle for classification
- Explain what the model actually "learns" during training (decision boundaries)
- Preview what lessons 01.1 will cover (data splitting)

Content Outline:

- Supervised learning recap: input features + labels → model → predictions
- What "fitting" a model means: adjusting internal parameters to minimize prediction errors
- The train/test paradox: you need data to train AND data to evaluate — so you split
- Why you can never evaluate on training data: the model has already seen it
- Overview of what comes next: 01.1 goes deep into the mechanics of splitting

Transitions: This lesson establishes the conceptual foundation. Lesson 01.1 dives into the practical steps of splitting a dataset before training begins.

Key Principles:

- Training is the model's "study session" — it only learns from the training set
- Evaluation must happen on data the model has never seen to be meaningful

Examples:

- Analogy: studying for an exam with practice problems (training) vs taking the actual exam (testing) — you can't practice ON the actual exam and then claim you learned from it
 - A decision tree "learns" by creating rules: "if petal length > 2.5 cm → not setosa"
-

01.1 Splitting Your Data

Learning Objectives:

- Apply `train_test_split` to divide a dataset into training and test sets
- Explain why a validation set (or cross-validation) improves generalization
- Avoid data leakage by fitting all preprocessing only on training data

Content Outline:

- **The split:** train set (80%) for learning, test set (20%) for evaluation
 - Why 80/20 is a reasonable starting split
 - Why random shuffling matters (ordered datasets would leak temporal patterns)
- **Stratification:** keeping the class distribution proportional in both sets
 - Critical when classes are imbalanced (e.g., 5% fraud, 95% legitimate)
- **Data leakage:** what it is and why it's devastating
 - Fitting a scaler on the full dataset before splitting → test data "leaks" into training normalization
 - Correct pattern: fit the scaler on training data only; transform both sets with it
- **Cross-validation** (concept): using multiple train/test splits and averaging results for more reliable estimates

Transitions: The data is split — now it's time to choose which algorithm will learn from the training set. Section 02 begins with an overview of the training and selection process.

Key Principles:

- Always split BEFORE any preprocessing that has fit parameters (scalers, encoders)
- Stratified split is almost always correct for classification — use it by default

Anti-patterns:

- **Evaluating on training data:** the model has memorized the answers; scores will be artificially perfect
- **Fitting the scaler on the full dataset:** test set statistics leak into training, inflating apparent performance

Examples:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

02.0 Selecting and Training

Learning Objectives:

- Describe the scikit-learn fit/predict API pattern
- Explain the role of hyperparameters vs parameters learned during training
- Preview the specific algorithms covered in 02.1 and the hands-on challenge in 02.2

Content Outline:

- The scikit-learn training pattern: `model = Classifier() → model.fit(X_train, y_train) → model.predict(X_test)`
- What "fit" does: the model adjusts its internal parameters to minimize errors on training data
- What "predict" does: applies learned parameters to new inputs to output a class label
- Parameters (learned, e.g. tree split points) vs hyperparameters (set by you, e.g. `max_depth`)
- Overview of what 02.1 covers: decision trees, logistic regression, random forests

Transitions: This overview frames the training API. Lesson 02.1 goes deeper into the algorithm choices — what each one does and when to pick it.

Key Principles:

- All scikit-learn classifiers follow the same `fit` / `predict` / `score` interface — switch algorithms by swapping the class
- Hyperparameters are choices YOU make; parameters are what the model learns from data

Examples:

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=5)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

02.1 Choosing a Classifier

Learning Objectives:

- Compare decision trees, logistic regression, and random forests on interpretability, speed, and accuracy
- Select the appropriate algorithm for a given classification scenario
- Recognize when to prefer a simpler model over a more powerful one

Content Outline:

- **Decision Tree**

- How it works: recursively splits features to minimize impurity (Gini or entropy)
- Strengths: interpretable, fast, handles non-linear boundaries
- Weaknesses: overfits easily without depth limits; sensitive to data changes
- When to use: quick baselines, interpretable outputs needed
- **Logistic Regression**
 - How it works: models probability of class membership using a sigmoid function
 - Strengths: interpretable coefficients, fast, probabilistic output
 - Weaknesses: assumes linear decision boundary; may underfit complex data
 - When to use: linearly separable problems, when probability estimates matter
- **Random Forest**
 - How it works: ensemble of decision trees; each tree votes; majority wins
 - Strengths: resists overfitting (diversity of trees), robust, generally accurate
 - Weaknesses: less interpretable, slower to train, more memory
 - When to use: general-purpose starting point when you don't know the data well

Transitions: Now that you know which algorithm to pick, lesson 02.2 puts it into practice — you'll train a classifier end-to-end and see predictions come out.

Key Principles:

- Start simple: a logistic regression or shallow decision tree is a better baseline than immediately reaching for a random forest
- Interpretability and accuracy trade off: the more powerful the model, the harder it is to explain

Anti-patterns:

- **Jumping to the most complex model first:** complex models are harder to debug when things go wrong
- **Never changing the algorithm:** no single algorithm wins on every dataset — try at least two

Examples:

Algorithm	Interpretable?	Handles Non-Linear?	Overfits?	Speed
Logistic Regression	✓ high	X no	X low	✓ fast
Decision Tree	✓ high	✓ yes	✓ high	✓ fast
Random Forest	X low	✓ yes	X low	~ medium

02.2 Training a Classifier

Challenge Prompt: Load the prepared dataset from `clean_data.csv`, split it into train and test sets (stratified, 80/20), train a `DecisionTreeClassifier` with `max_depth=5`, and print the accuracy score on the test set.

Solution Code:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv("clean_data.csv")
X = df.drop("label", axis=1)
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = DecisionTreeClassifier(max_depth=5, random_state=42)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, predictions):.4f}")

```

Challenge Code:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv("clean_data.csv")
X = df.drop("label", axis=1)
y = df["label"]

# Step 1: split the data (80% train, 20% test, stratified, random_state=42)
X_train, X_test, y_train, y_test = # your code here

# Step 2: create a DecisionTreeClassifier with max_depth=5 and
random_state=42
model = # your code here

# Step 3: train the model
# your code here

# Step 4: predict on the test set and print accuracy
predictions = # your code here
print(f"Accuracy: {accuracy_score(y_test, predictions):.4f}")

```

03.0 Evaluating Performance

Learning Objectives:

- Explain why accuracy alone can be misleading for classification
- Interpret a confusion matrix to identify what types of errors a model is making
- Preview the metric deep-dive in lesson 03.1

Content Outline:

- **The accuracy trap:** when classes are imbalanced, high accuracy means nothing
 - Example: 95% of transactions are legitimate → a model predicting "legitimate" always scores 95% accuracy, catching zero fraud
- **Confusion matrix:** a table showing true positives, true negatives, false positives, false negatives
 - True Positive (TP): correctly predicted positive
 - False Positive (FP): predicted positive but was actually negative (Type I error)
 - False Negative (FN): predicted negative but was actually positive (Type II error)
 - True Negative (TN): correctly predicted negative
- Why error type matters: in medical diagnosis, a false negative (missing a disease) is far worse than a false positive

Transitions: This overview explains WHY we need more than accuracy. Lesson 03.1 goes into precision, recall, F1, and how to read scikit-learn's classification report.

Key Principles:

- Never report only accuracy for imbalanced datasets — always show the confusion matrix
- The cost of different error types is domain-specific: you must understand the business context to choose the right metric

Examples:

Predicted →	Positive	Negative
Actual Positive	TP (90)	FN (10)
Actual Negative	FP (5)	TN (895)

03.1 Classification Reports

Learning Objectives:

- Calculate precision, recall, and F1-score from a confusion matrix
- Interpret scikit-learn's `classification_report` output
- Recognize signs of overfitting from train vs test score discrepancy

Content Outline:

- **Precision:** of all instances predicted positive, what fraction were actually positive?
 - $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
 - High precision → few false alarms
- **Recall:** of all actual positives, what fraction did the model catch?
 - $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

- High recall → few missed positives
- **F1-score:** harmonic mean of precision and recall — useful when you need to balance both
 - $F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$
- **scikit-learn's `classification_report`:** outputs precision, recall, F1, and support for each class
- **Overfitting check:** compare train accuracy vs test accuracy
 - Train: 98%, Test: 72% → classic overfitting — model memorized training data
 - Train: 82%, Test: 80% → healthy generalization gap

Transitions: With evaluation covered, the assessment checks understanding of the full training pipeline from split to evaluation.

Key Principles:

- Choose the metric that penalizes the worst error type for your domain
- Always compare training and test scores — a large gap signals overfitting

Examples:

```
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(y_test, predictions))
print(confusion_matrix(y_test, predictions))
```

Sample output:

	precision	recall	f1-score	support
0	0.91	0.95	0.93	150
1	0.88	0.80	0.84	75
accuracy			0.90	225

04.0 Classification Training Assessment 🧠

Learning Objectives:

- Demonstrate understanding of the complete classification training pipeline
- Correctly apply split, fit, predict, and evaluate patterns
- Identify common pitfalls in model training and evaluation

Question Format: Multiple choice

Questions:

1. You have a dataset with classes: 90% Class A, 10% Class B. A model predicts Class A for every sample. What accuracy does it achieve?
 - a) 10%

- b) 50%
 - c) 90%
 - d) 100%
2. What does `model.fit(X_train, y_train)` do?
- a) Generates predictions for `X_train`
 - b) Adjusts the model's internal parameters to minimize errors on the training set
 - c) Evaluates the model's accuracy on `y_train`
 - d) Splits `X_train` into smaller training batches
3. Why must you use `stratify=y` when calling `train_test_split` for a classification problem?
- a) To ensure both sets have the same number of samples
 - b) To preserve the class distribution across train and test sets
 - c) To randomly shuffle the dataset before splitting
 - d) To avoid duplicates in the test set
4. Your model scores 97% on training data but only 61% on test data. What does this indicate?
- a) The model is underfitting — it needs more features
 - b) The model is overfitting — it memorized training data and fails to generalize
 - c) The test set is too small — results are unreliable
 - d) The split was not stratified — training and test sets are imbalanced
5. You fit a `MinMaxScaler` on the full dataset BEFORE splitting. What problem does this cause?
- a) The scaler will fail to transform the test set
 - b) Test set statistics leak into the training normalization, artificially inflating test scores
 - c) The model will converge more slowly during training
 - d) No problem — fitting on the full dataset gives better normalization
6. Which metric best captures model performance when you care more about catching all positives than avoiding false alarms?
- a) Accuracy
 - b) Precision
 - c) Recall
 - d) Specificity
7. Which of the following is NOT a valid reason to choose logistic regression over a random forest?
- a) You need interpretable feature coefficients
 - b) You expect a linear decision boundary
 - c) Training speed is a priority
 - d) You want the highest possible accuracy on complex non-linear data

Evaluation Criteria:

- 7/7: Excellent — complete mastery of the training pipeline
- 5–6/7: Good — minor gaps in metrics or split mechanics

- 3–4/7: Needs review — revisit sections 01 and 03
- Below 3/7: Restart recommended — core training concepts not yet solid

Score Interpretation: Questions map to: imbalanced accuracy trap (Q1), what fit does (Q2), stratification (Q3), overfitting (Q4), data leakage (Q5), recall vs precision (Q6), algorithm selection (Q7).

05.0 Your Classifier Is Trained

Content Outline:

- Course recap: what students accomplished
 - Understood the supervised learning loop for classification
 - Split a dataset correctly (stratified, leakage-free)
 - Trained three classifier types and chose between them
 - Evaluated with accuracy, confusion matrix, precision, recall, and F1
- Connection to bigger picture: training a classifier is the first half — debugging and improving a poorly-performing model is the second half, covered in upcoming modules
- Next steps: the next module applies the same training mechanics to regression (predicting a number instead of a category)
- Resources for further exploration:
 - scikit-learn documentation: classifiers, [train_test_split](#), [classification_report](#)
 - "Hands-On Machine Learning with Scikit-Learn and TensorFlow" (Aurélien Géron) — chapters on classification
 - Kaggle: practice classification on real competition datasets
- Encouragement: training your first classifier from scratch is the moment the theory becomes real — every ML engineer started exactly here

Note: This is conclusion only — no theory, exercises, or assessment.
